15.095 Machine Learning HW3

Chris Low

Question 1

(a)

False.

Even if every tree is allowed to consider all p features at every split, the trees are still trained on different bootstrap samples of the training data. Since bootstrap sampling is done with replacement, some observations may appear multiple times while others may not appear at all. Therefore, different trees may choose different optimal splits and need not be identical. Thus, the forest's majority-vote prediction need not be the same as the prediction of a single CART tree.

For example, suppose the original training observations are $\{1, 2, 3, 4, 5\}$. Two bootstrap samples could be $\{2, 3, 1, 2, 5\}$ and $\{3, 1, 4, 5, 1\}$.

The first sample omits observation 4 and repeats observation 2, while the second omits observation 2 and repeats observation 1. Hence, even when all p features are available at every split, the two trees are trained on different data and may select different splits.

(b)

True.

Since feature j only takes positive values, $\log(\cdot)$ is strictly increasing. This means that taking the log changes the numerical values of the feature, but does not change their ordering. Therefore, for any threshold $b > 0$,

$$x_{ij} < b \iff \log x_{ij} < \log b.$$

So if the original OCT has a split $x_{ij} < b$, we can replace it with $\log x_{ij} < \log b$ after the transformation. The exact same observations will still go to the left and right sides of the split.

This also works in the reverse direction. If the transformed tree has a split $\log x_{ij} < b'$, then we can use $x_{ij} < e^{b'}$ on the original data. Thus, any split on one dataset has an equivalent split on the other dataset.

As a result, we can construct the same tree on both datasets, with the same observations in each leaf, the same leaf sizes, the same number of splits, and the same number of misclassified observations. The OCT objective is

$$\min \sum_{t \in \mathcal{L}} L_t + \alpha \sum_{t \in \mathcal{B}} d_t,$$

where the first term represents misclassification error and the second term penalizes the number of splits. Since neither of these changes under the log transformation, the optimal objective value is the same for the original and transformed data.

(c)

True.

Since $z_{it} = 1$ if observation i is assigned to leaf t, the total number of training observations assigned to leaf t is $\sum_{i=1}^{n} z_{it}$.

We want every leaf that predicts class A to contain at least N_A training observations. Since $c_{At} = 1$ if leaf t predicts class A, we can add the constraint

$$\sum_{i=1}^{n} z_{it} \geq N_A c_{At}, \qquad \forall t \in \mathcal{L}.$$

If leaf t predicts class A, then $c_{At} = 1$, and the constraint becomes $\sum_{i=1}^{n} z_{it} \geq N_A$, so at least N_A training observations must be assigned to that leaf. If leaf t does not predict class A, then $c_{At} = 0$, and the constraint becomes $\sum_{i=1}^{n} z_{it} \geq 0$, which adds no additional restriction. Therefore, this requirement can be added using one linear constraint per leaf and no additional variables.

(d)

False.

The CART prediction is the average demand of the observations in the leaf:

$$\hat{y} = \frac{40 + 50 + 60 + 70 + 130}{5} = 70.$$

However, the weighted-cost prescription does not simply set $z = \hat{y}$. Since all five observations are in the same CART leaf, they each receive weight $1/5$. Therefore, the prescription solves

$$\min_{z \geq 0} \frac{1}{5} \sum_{i=1}^{5} \left(7z - 10 \min(z, y_i)\right).$$

To find the optimal z, consider the effect of baking one additional loaf. If demand is greater than the number of loaves baked, $y > z$, then all loaves are sold and $c(z; y) = 7z - 10z = -3z$. Thus, each additional loaf decreases cost by 3 (equivalently, increases profit by 3).

On the other hand, if demand is less than the number of loaves baked, $y < z$, then only y loaves are sold and $c(z; y) = 7z - 10y$. Since y is fixed, increasing z by one increases cost by 7. Therefore, an additional loaf contributes -3 to cost if it is sold and $+7$ if it is not sold.

For $z < 40$, the additional loaf would be sold on all five historical days, so the slope of the total cost is

$$5(-3) = -15 < 0.$$

For $40 < z < 50$, the additional loaf would be unsold on one day and sold on four days, giving

$$7 + 4(-3) = -5 < 0.$$

For $50 < z < 60$, it would be unsold on two days and sold on three days, giving

$$2(7) + 3(-3) = 5 > 0.$$

Thus, the objective decreases up to $z = 50$ and increases after $z = 50$, so $z^* = 50$.

Therefore, although the CART tree predicts demand of 70, the weighted-cost prescription bakes 50 loaves, not 70.

(e)

False.

Under inverse propensity weighting, the estimated reward for point i under treatment t is

$$\Gamma_{it} = \mathbf{1}\{T_i = t\}\frac{y_i}{p_{it}}.$$

Therefore, for the treatment that point i actually received,

$$\Gamma_{iT_i} = \frac{y_i}{p_{iT_i}},$$

while for every other treatment $t \neq T_i$, we have $\Gamma_{it} = 0$.

If each training point is placed in its own leaf, the policy chooses the treatment with the largest estimated reward. Since $y_i \in \mathbb{R}$, it is possible that $y_i < 0$. Because $p_{iT_i} > 0$, this would imply

$$\Gamma_{iT_i} = \frac{y_i}{p_{iT_i}} < 0,$$

while every treatment not received has reward 0.

For example, suppose point i received treatment 1, with $y_i = -5$ and $p_{i1} = 0.5$.

Then $\Gamma_{i1} = \frac{-5}{0.5} = -10$, while for another treatment, say treatment 2, $\Gamma_{i2} = 0$.

Since the policy maximizes reward and $0 > -10$, it would prescribe treatment 2 rather than the treatment point i actually received. Therefore, the tree does not necessarily prescribe every training point its observed treatment.

Question 2

(a)

At the root, the four possible splits are $x_1 < 1$, $x_1 < 2$, $x_2 < 1$, and $x_2 < 2$. For each child node t, the Gini impurity is

$$I_t = 1 - \sum_{k \in \{A,B\}} p_{kt}^2,$$

and the weighted Gini of a split is

$$\frac{n_L}{n} I_L + \frac{n_R}{n} I_R,$$

where $n = 31$.

For the split $x_1 < 1$, the left child contains $(A, B) = (6, 4)$ and the right child contains $(A, B) = (13, 8)$. Thus,

$$I_L = 1 - \left(\frac{6}{10}\right)^2 - \left(\frac{4}{10}\right)^2 = 0.48,$$

$$I_R = 1 - \left(\frac{13}{21}\right)^2 - \left(\frac{8}{21}\right)^2 \approx 0.4717,$$

$$G(x_1 < 1) = \frac{10}{31}(0.48) + \frac{21}{31}(0.4717) \approx 0.4743.$$

For the split $x_1 < 2$, the left child contains $(A, B) = (10, 7)$ and the right child contains $(A, B) = (9, 5)$. Thus,

$$I_L = 1 - \left(\frac{10}{17}\right)^2 - \left(\frac{7}{17}\right)^2 \approx 0.4844,$$

$$I_R = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \approx 0.4592,$$

$$G(x_1 < 2) = \frac{17}{31}(0.4844) + \frac{14}{31}(0.4592) \approx 0.4730.$$

For the split $x_2 < 1$, the left child contains $(A, B) = (2, 5)$ and the right child contains $(A, B) = (17, 7)$. Thus,

$$I_L = 1 - \left(\frac{2}{7}\right)^2 - \left(\frac{5}{7}\right)^2 \approx 0.4082,$$

$$I_R = 1 - \left(\frac{17}{24}\right)^2 - \left(\frac{7}{24}\right)^2 \approx 0.4132,$$

$$G(x_2 < 1) = \frac{7}{31}(0.4082) + \frac{24}{31}(0.4132) \approx 0.4121.$$

For the split $x_2 < 2$, the left child contains $(A, B) = (14, 6)$ and the right child contains $(A, B) = (5, 6)$. Thus,

$$I_L = 1 - \left(\frac{14}{20}\right)^2 - \left(\frac{6}{20}\right)^2 = 0.42,$$

$$I_R = 1 - \left(\frac{5}{11}\right)^2 - \left(\frac{6}{11}\right)^2 \approx 0.4959,$$

$$G(x_2 < 2) = \frac{20}{31}(0.42) + \frac{11}{31}(0.4959) \approx 0.4469.$$

Therefore, the four weighted Gini impurities are

$$\begin{aligned} G(x_1 < 1) &\approx 0.4743, \\ G(x_1 < 2) &\approx 0.4730, \\ G(x_2 < 1) &\approx 0.4121, \\ G(x_2 < 2) &\approx 0.4469. \end{aligned}$$

Since CART chooses the split with the smallest weighted Gini impurity, the root split is $\boxed{x_2 < 1}$.

After choosing $x_2 < 1$ as the root split, we split each child separately.

Left child: $x_2 < 1$

This node contains $(A, B) = (2, 5)$, so $n = 7$. The two useful candidate splits are $x_1 < 1$ and $x_1 < 2$.

For $x_1 < 1$, the left child contains $(A, B) = (1, 0)$ and the right child contains $(A, B) = (1, 5)$. Thus,

$$I_L = 0,$$

$$I_R = 1 - \left(\frac{1}{6}\right)^2 - \left(\frac{5}{6}\right)^2 \approx 0.2778.$$

$$G(x_1 < 1) = \frac{1}{7}(0) + \frac{6}{7}(0.2778) \approx 0.2381.$$

For $x_1 < 2$, the left child contains $(A, B) = (2, 0)$ and the right child contains $(A, B) = (0, 5)$. Both are pure, so

$$I_L = I_R = 0,$$

$$G(x_1 < 2) = 0.$$

$$\boxed{x_1 < 2}$$

for the left child.

The resulting leaves are $(A, B) = (2, 0)$, which predicts class A, and $(A, B) = (0, 5)$, which predicts class B. Both leaves have zero misclassification error.

Right child: $x_2 \geq 1$

This node contains $(A, B) = (17, 7)$, so $n = 24$. The useful candidate splits are $x_1 < 1$, $x_1 < 2$, and $x_2 < 2$.

For $x_1 < 1$, the two children contain $(A, B) = (5, 4)$ and $(A, B) = (12, 3)$. Thus,

$$I_L = 1 - \left(\frac{5}{9}\right)^2 - \left(\frac{4}{9}\right)^2 \approx 0.4938,$$

$$I_R = 1 - \left(\frac{12}{15}\right)^2 - \left(\frac{3}{15}\right)^2 = 0.32.$$

$$G(x_1 < 1) = \frac{9}{24}(0.4938) + \frac{15}{24}(0.32) \approx 0.3852.$$

For $x_1 < 2$, the two children contain $(A, B) = (8, 7)$ and $(A, B) = (9, 0)$. Thus,

$$I_L = 1 - \left(\frac{8}{15}\right)^2 - \left(\frac{7}{15}\right)^2 \approx 0.4978,$$

$$I_R = 0.$$

$$G(x_1 < 2) = \frac{15}{24}(0.4978) + \frac{9}{24}(0) \approx 0.3111.$$

For $x_2 < 2$, the two children contain $(A, B) = (12, 1)$ and $(A, B) = (5, 6)$. Thus,

$$I_L = 1 - \left(\frac{12}{13}\right)^2 - \left(\frac{1}{13}\right)^2 \approx 0.1420,$$

$$I_R = 1 - \left(\frac{5}{11}\right)^2 - \left(\frac{6}{11}\right)^2 \approx 0.4959.$$

$$G(x_2 < 2) = \frac{13}{24}(0.1420) + \frac{11}{24}(0.4959) \approx 0.3042.$$

Comparing the three candidate splits,

$$\begin{aligned} G(x_1 < 1) &\approx 0.3852, \\ G(x_1 < 2) &\approx 0.3111, \\ G(x_2 < 2) &\approx 0.3042. \end{aligned}$$

Therefore, CART chooses $\boxed{x_2 < 2}$ for the right child.

The leaf with $1 \le x_2 < 2$ contains $(A, B) = (12, 1)$, so it predicts class A and makes 1 error. The leaf with $x_2 \ge 2$ contains $(A, B) = (5, 6)$, so it predicts class B and makes 5 errors.

Thus, the four final leaves make $0 + 0 + 1 + 5 = 6$ total errors, so the training error of the greedy CART tree is $\boxed{6}$.

(b)

Instead of choosing the root greedily, we fix each possible root split and find the best second-level split for each of its two children. The minimum total errors obtained for the four possible root splits are

$$\begin{aligned} x_1 < 1: &\quad 1 + 4 = 5, \\ x_1 < 2: &\quad 1 + 0 = 1, \\ x_2 < 1: &\quad 0 + 6 = 6, \\ x_2 < 2: &\quad 3 + 0 = 3, \end{aligned}$$

where the two numbers in each sum are the minimum errors from the left and right subtrees, respectively.

The smallest total error is obtained by choosing $\boxed{x_1 < 2}$ as the root.

For the left branch, $x_1 < 2$, the best split is $x_2 < 2$. This gives $x_2 < 2 : (A, B) = (10, 1)$, which predicts class A and makes one error, and $x_2 \ge 2 : (A, B) = (0, 6)$, which predicts class B and makes no errors.

For the right branch, $x_1 \ge 2$, the best split is $x_2 < 1$. This gives $x_2 < 1 : \quad (A, B) = (0, 5)$, which predicts class B and makes no errors, and $x_2 \ge 1 : \quad (A, B) = (9, 0)$, which predicts class A and also makes no errors.

Therefore, the total error is $\boxed{1}$.

From part (a), the weighted Gini impurities of the four possible root splits were

$$\begin{aligned} G(x_2 < 1) &= 0.4121, \\ G(x_2 < 2) &= 0.4469, \\ G(x_1 < 2) &= 0.4730, \\ G(x_1 < 1) &= 0.4743. \end{aligned}$$

Thus, $x_1 < 2$, which gives the globally best tree, was only the third-best root split according to Gini impurity.

If the root splits are instead evaluated by their immediate misclassification errors, they give

$$\begin{aligned} x_2 < 1: &\quad 9, \\ x_2 < 2: &\quad 11, \\ x_1 < 1: &\quad 12, \\ x_1 < 2: &\quad 12. \end{aligned}$$

Thus, $x_1 < 2$ is actually tied for the worst root split when only the immediate misclassification error is considered.

CART cannot find the optimal tree because it grows the tree greedily. At each step, it chooses the split that looks best immediately and does not consider how that split affects later splits. Although $x_1 < 2$ does not perform well at the root by itself, it creates two branches that can be separated very well by the next splits. Therefore, the locally best split does not necessarily lead to the globally best tree.

Finally, no tree of any depth using the allowed integer thresholds can achieve an error below 1. The cell $\boxed{(0,1) \times (1,2)}$ contains 5 class-A patients and 1 class-B patient. Since integer-threshold splits cannot separate observations within the same unit cell, at least one patient in this cell must always be misclassified. Since the tree above achieves error 1, it is globally optimal.

(c)

(i.)

The objective is error$(T) + \alpha \cdot$ (number of splits in T). From the previous parts, the best achievable classification errors using $0, 1, 2$, and 3 splits are

$$E_0 = 12, \qquad E_1 = 9, \qquad E_2 = 6, \qquad E_3 = 1.$$

Therefore, adding the complexity penalty α for each split gives the four possible objective values

$$\begin{aligned} J_0(\alpha) &= 12, \\ J_1(\alpha) &= 9 + \alpha, \\ J_2(\alpha) &= 6 + 2\alpha, \\ J_3(\alpha) &= 1 + 3\alpha. \end{aligned}$$

The three-split tree remains optimal until its objective reaches that of the tree with no splits. Setting these equal gives $1 + 3\alpha = 12$,

so $\alpha = \frac{11}{3}$.

The one- and two-split trees do not become optimal before this point, and for $\alpha > 11/3$ they are both more expensive than the no-split tree. Therefore, the optimal tree is

$$\begin{cases} 0 \le \alpha < \dfrac{11}{3}: & \text{the three-split tree from part (b), with objective } 1 + 3\alpha, \\ \alpha = \dfrac{11}{3}: & \text{the three-split tree and the no-split tree tie, with objective 12,} \\ \alpha > \dfrac{11}{3}: & \text{the no-split tree, with objective 12.} \end{cases}$$

Thus, trees with 1 or 2 splits are never the unique optimum (in fact, they are never optimal for any α).

(ii.)

We now start with the greedy CART tree from part (a) and apply cost-complexity pruning. The full CART tree has three splits and a total classification error of 6, so its objective is

$$J_3^{\mathrm{CART}}(\alpha) = 6 + 3\alpha.$$

For the two-split tree, the best choice is to prune the right child split $x_2 < 2$. The two leaves on this branch, $(A, B) = (12, 1)$ and $(A, B) = (5, 6)$, are merged into one leaf containing $(A, B) = (17, 7)$.

This leaf predicts class A and therefore makes 7 errors. The left subtree still has zero errors, so the best two-split pruned tree has total error 7 and objective

$$J_2^{\mathrm{CART}}(\alpha) = 7 + 2\alpha.$$

Pruning again leaves only the root split $x_2 < 1$. The left leaf contains $(A, B) = (2, 5)$ and therefore makes 2 errors, while the right leaf contains $(A, B) = (17, 7)$ and makes 7 errors. Hence, the one-split tree has $2 + 7 = 9$ errors and objective

$$J_1^{\mathrm{CART}}(\alpha) = 9 + \alpha.$$

Finally, pruning the root gives a tree with no splits. Since the full dataset contains 19 class-A and 12 class-B observations, the tree predicts A for everyone and makes 12 errors. Thus,

$$J_0^{\mathrm{CART}}(\alpha) = 12.$$

Therefore, the four objectives available through pruning are

$$\begin{aligned} J_3^{\mathrm{CART}}(\alpha) &= 6 + 3\alpha, \\ J_2^{\mathrm{CART}}(\alpha) &= 7 + 2\alpha, \\ J_1^{\mathrm{CART}}(\alpha) &= 9 + \alpha, \\ J_0^{\mathrm{CART}}(\alpha) &= 12. \end{aligned}$$

The first transition occurs when $6 + 3\alpha = 7 + 2\alpha$, which gives $\alpha = 1$.

The next transition occurs when $7 + 2\alpha = 9 + \alpha$, which gives $\alpha = 2$.

Finally, $9 + \alpha = 12$ gives $\alpha = 3$.

Therefore, CART returns

$$\begin{cases} 0 \le \alpha < 1 & : \text{the full three-split CART tree,} \\ \alpha = 1 & : \text{the three- and two-split trees tie,} \\ 1 < \alpha < 2 & : \text{the two-split pruned tree,} \\ \alpha = 2 & : \text{the two- and one-split trees tie,} \\ 2 < \alpha < 3 & : \text{the one-split tree,} \\ \alpha = 3 & : \text{the one- and zero-split trees tie,} \\ \alpha > 3 & : \text{the zero-split tree.} \end{cases}$$

From part (i), the globally optimal objective is

$$J^*(\alpha) = \begin{cases} 1 + 3\alpha, & 0 \le \alpha \le \frac{11}{3}, \\ 12, & \alpha \ge \frac{11}{3}. \end{cases}$$

For $\alpha < 11/3$, the globally optimal tree has a strictly smaller objective than the tree returned by CART. Once $\alpha \ge 11/3$, the globally optimal solution has no splits and objective 12, which CART can also obtain by pruning the entire tree.

Therefore, CART achieves the same objective value as the globally optimal tree exactly when

$$\boxed{\alpha \geq \frac{11}{3}}.$$

(d)

(i.)

For each feature $j \in [p]$ and leaf $t \in \mathcal{L}$, introduce a continuous variable

$$u_{jt} \in [0, 1],$$

where u_{jt} indicates whether feature j is tested at least once on the path from the root to leaf t.

For example, suppose the path to leaf t contains two nodes, and both nodes split on feature x_2. Then

$$a_{2,1} = 1 \qquad \text{and} \qquad a_{2,2} = 1,$$

since a_{jm} indicates whether feature j is used at a specific node m. However,

$$u_{2t} = 1,$$

since u_{jt} only indicates whether feature j is used at least once anywhere on the path to leaf t. Thus, even though x_2 is used twice, its cost is counted only once.

For every ancestor $m \in P_t$, we impose

$$u_{jt} \geq a_{jm}, \qquad \forall j \in [p],\ t \in \mathcal{L},\ m \in P_t.$$

Thus, if any node on the path to leaf t splits on feature j, then u_{jt} is forced to 1. The budget constraint for each leaf is then

$$\sum_{j=1}^{p} c_j u_{jt} \leq C, \qquad \forall t \in \mathcal{L}.$$

There are $p|\mathcal{L}|$ additional continuous variables because for each of the $|\mathcal{L}|$ leaves, we need one variable for each of the p possible features. For example, if $p = 2$ and there are four leaves, we need

$$u_{1,t_1}, u_{2,t_1}, u_{1,t_2}, u_{2,t_2}, u_{1,t_3}, u_{2,t_3}, u_{1,t_4}, u_{2,t_4},$$

for a total of $2(4) = 8$ variables. These variables record whether each feature is used on the path to each leaf. They do not imply that every feature is actually used.

Each leaf has at most D ancestor nodes. Therefore, for each feature-leaf pair (j, t), there are at most D linking constraints of the form $u_{jt} \geq a_{jm}$. Since there are $p|\mathcal{L}|$ feature-leaf pairs, there are at most $pD|\mathcal{L}|$ linking constraints.

In addition, there is one budget constraint for each leaf, giving another $|\mathcal{L}|$ constraints. Therefore, the total number of additional constraints is at most

$$pD|\mathcal{L}| + |\mathcal{L}| = \boxed{|\mathcal{L}|(pD + 1).}$$

(ii.)

We have

$$c_1 = 1, \qquad c_2 = 3, \qquad C = 3.$$

Since using both features on the same path would cost $c_1 + c_2 = 1 + 3 = 4 > C$, each path can use only x_1 or only x_2. An x_1-only tree has minimum error 12 because no matter how we split by x_1 ($x_1 < 1$ or $x_1 < 2$), we would get an A majority. Using only x_2, we can split first at $x_2 < 1$ and then split the right child at $x_2 < 2$. The three leaves have class counts

$$(2, 5), \qquad (12, 1), \qquad (5, 6),$$

so their respective errors are 2, 1, and 5. Thus, the total error is $2 + 1 + 5 = 8$. Therefore, the minimum error under the correct budget constraint is $\boxed{8}$.

Under the colleague's constraint,

$$\sum_{m \in P_t} \sum_{j=1}^{p} c_j a_{jm} \leq C,$$

a feature is charged every time it is tested. Thus, a path that tests x_2 twice is charged $3 + 3 = 6 > C$, so the two-split x_2 tree above is no longer feasible. The best feasible tree instead uses the single split $x_2 < 1$. Its two leaves contain $(2, 5)$ and $(17, 7)$ observations, giving $2 + 7 = 9$ errors. Hence, the smallest error under the colleague's constraint is $\boxed{9}$.

The two answers differ because the colleague's constraint charges for every use of a feature, while the correct formulation charges each feature only once along a path. Thus, the colleague's constraint is too restrictive and rules out the x_2-only tree with error 8.

In [1]:
```
using CSV, DataFrames, Statistics, CategoricalArrays, Plots
isdefined(Main, :IAI) || error("IAI is not loaded: switch to the Julia kernel that uses the IAI syste

const SEED = 15095
df = CSV.read("subscribers.csv", DataFrame; types = Dict(:snapshot_month => String))
println(size(df))
```

```
(54331
, 15)
```

Part (a)

Training months

In [2]:
```
by_month = sort(combine(groupby(df, :snapshot_month),
    :churned_60d     => mean    => :churn_rate,
    :active_days_30d => mean    => :mean_active_days,
    :active_days_30d => maximum => :max_active_days), :snapshot_month)
churn_billing = sort(unstack(
    combine(groupby(df, [:snapshot_month, :billing]), :churned_60d => mean => :churn_rate),
    :snapshot_month, :billing, :churn_rate), :snapshot_month)

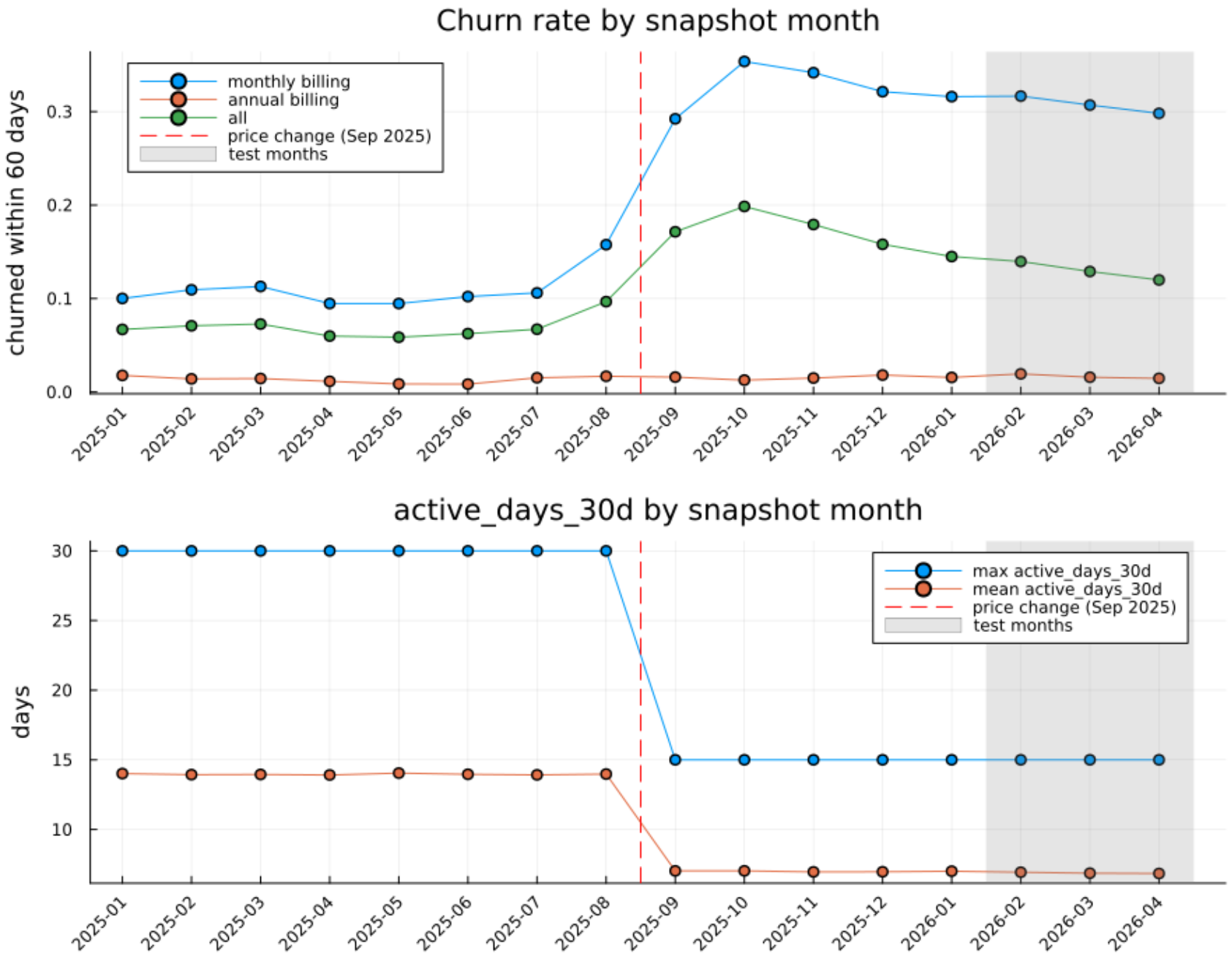
months = by_month.snapshot_month
xs = 1:length(months)
i_sep = findfirst(==("2025-09"), months)   # first post-price-change snapshot
i_feb = findfirst(==("2026-02"), months)   # first test month

p1 = plot(xs, [churn_billing.monthly churn_billing.annual by_month.churn_rate],
          label = ["monthly billing" "annual billing" "all"], marker = :circle,
          ylabel = "churned within 60 days", title = "Churn rate by snapshot month")
p2 = plot(xs, [by_month.max_active_days by_month.mean_active_days],
          label = ["max active_days_30d" "mean active_days_30d"], marker = :circle,
          ylabel = "days", title = "active_days_30d by snapshot month")
for p in (p1, p2)
    vline!(p, [i_sep - 0.5], color = :red, linestyle = :dash, label = "price change (Sep 2025)")
    vspan!(p, [i_feb - 0.5, length(months) + 0.5], color = :gray, alpha = 0.2, label = "test months")
```

```
    plot!(p, xticks = (xs, months), xrotation = 45)
end
fig_months = plot(p1, p2, layout = (2, 1), size = (900, 700), left_margin = 5Plots.mm, bottom_margin
savefig(fig_months, "churn_by_month.png")

before, after = df.snapshot_month .<= "2025-08", df.snapshot_month .>= "2025-09"
monthly = df.billing .== "monthly"
println("monthly-billing churn rate: Jan–Aug 2025 = ", round(mean(df.churned_60d[before .& monthly]),
        ", Sep 2025–Apr 2026 = ", round(mean(df.churned_60d[after .& monthly]), digits = 3))
println("max active_days_30d: Jan–Aug 2025 = ", maximum(df.active_days_30d[before]),
        ", Sep 2025–Apr 2026 = ", maximum(df.active_days_30d[after]))
display("image/png", fig_months)
```

```
monthly-billing churn rate: Jan–Aug 2025 = 0.11, Sep 2025–Apr 2026 = 0.32
max active_days_30d: Jan–Aug 2025 = 30, Sep 2025–Apr 2026 = 15
```



Training months. I use September 2025 through January 2026 for training. September marks a clear regime change:

monthly-billing churn increases from 11% in January–August 2025 to 32% afterward, while the maximum active_days_30d falls from 30 to 15. I therefore use the post-change months for training and reserve February–April 2026 as the test set.f

Keep/drop evidence

In [3]:
```
r3(x) = round(x, digits = 3)
pct(x) = string(round(100x, digits = 1), "%")
churn_by(col) = combine(groupby(df, col), :churned_60d => mean => :rate)
rng(col) = (t = churn_by(col); "$(pct(minimum(t.rate)))–$(pct(maximum(t.rate)))")

# last_invoice_usd: same value in every row of a subscriber -> read at extraction (30 Jun 2026), not
subs = combine(groupby(df, :subscriber_id),
    :last_invoice_usd => (x -> length(unique(x)) == 1) => :constant,
    :last_invoice_usd => first                         => :inv,
    :churned_60d      => maximum                       => :ever_churned)
list_price = in.(subs.inv, Ref(Set([20.0, 25.0, 240.0, 300.0])))
ever_zero, ever_other = mean(subs.ever_churned[subs.inv .== 0]), mean(subs.ever_churned[.!list_price

# signup_channel: age mix of student_promo
promo_young = mean(df.age_band[df.signup_channel .== "student_promo"] .== "18–24")

tenure_rate(m) = mean(df.churned_60d[m])
fp = df.failed_payment_30d .== 1

keep_drop = DataFrame(
    column = ["subscriber_id", "snapshot_month", "age_band", "signup_channel", "last_invoice_usd",
              "billing", "tenure_months", "region",
              "active_days_30d, usage_trend, features_used, days_since_last_login",
              "support_tickets_90d, failed_payment_30d", "churned_60d"],
    decision = ["drop", "drop", "drop", "drop", "drop",
                "keep", "keep", "keep", "keep", "keep", "label"],
    evidence = [
        "corr with tenure_months = $(r3(cor(df.subscriber_id, df.tenure_months)))",
        "time index; monthly-billing churn jumps at Sep 2025 (plot above)",
        "company policy; churn by band $(rng(:age_band))",
        "student_promo is $(pct(promo_young)) age 18–24",
```

```
        "constant across all rows for $(pct(mean(subs.constant))) of subscribers; ever churned: $(sum
        "churn: annual $(pct(mean(df.churned_60d[.!monthly]))) vs monthly $(pct(mean(df.churned_60d[m
        "churn: tenure ≤ 3 months $(pct(tenure_rate(df.tenure_months .<= 3))) vs > 24 months $(pct(te
        "churn by region $(rng(:region))",
        "usage measured in the 30 days before the snapshot",
        "churn: failed payment $(pct(mean(df.churned_60d[fp]))) vs none $(pct(mean(df.churned_60d[.!f
        "target"])
```

Out[3]: 11×3 DataFrame

Row	column	decision	evidence
	String	String	String
1	subscriber_id	drop	corr with tenure_months = -0.951
2	snapshot_month	drop	time index; monthly-billing churn jumps at Sep 2025 (plot above)
3	age_band	drop	company policy; churn by band 10.7%–12.0%
4	signup_channel	drop	student_promo is 94.2% age 18–24
5	last_invoice_usd	drop	constant across all rows for 100.0% of subscribers; ever churned: 1 of 2291 at list price vs 95.2% at $0 and 91.6% other
6	billing	keep	churn: annual 1.5% vs monthly 19.7%
7	tenure_months	keep	churn: tenure ≤ 3 months 23.0% vs > 24 months 7.0%
8	region	keep	churn by region 10.8%–11.2%
9	active_days_30d, usage_trend, features_used, days_since_last_login	keep	usage measured in the 30 days before the snapshot
10	support_tickets_90d, failed_payment_30d	keep	churn: failed payment 31.4% vs none 10.3%
11	churned_60d	label	target

We drop `subscriber_id` because it is an account identifier rather than a meaningful predictive feature; its strong correlation with tenure ($r = -0.951$) also suggests that it encodes information about when a subscriber joined. We drop `snapshot_month` because it is a time index that directly captures the pre- versus post-policy regime rather

than subscriber behavior. We exclude `age_band` because company policy prohibits targeting based on age, and we also exclude `signup_channel` because it can act as a strong proxy for age: 94.2% of subscribers from `student_promo` are ages 18--24. Finally, we drop `last_invoice_usd` because it is constant across all monthly snapshots for each subscriber, suggesting that it was measured at extraction rather than at the historical snapshot and could therefore introduce leakage.

Features and splits

We construct the full and kept-feature datasets and reserve February–April 2026 as the test set. The two OCTs below use all observations through January 2026 as required.

In [4]:
```
const CAT_COLS = ["billing", "region", "age_band", "signup_channel"]

all_cols  = setdiff(names(df), ["snapshot_month", "churned_60d"])   # model (1)
kept_cols = ["billing", "tenure_months", "region",
             "active_days_30d", "usage_trend", "features_used",
             "support_tickets_90d", "failed_payment_30d", "days_since_last_login"]

# Build feature tables once on the full data so categorical levels are shared across splits
function make_X(d, cols)
    X = select(d, cols)
    for c in intersect(cols, CAT_COLS)
        X[!, c] = categorical(X[!, c])
    end
    return X
end
X_all  = make_X(df, all_cols)
X_kept = make_X(df, kept_cols)
y      = df.churned_60d

test_mask  = df.snapshot_month .>= "2026-02"
upto_jan   = df.snapshot_month .<= "2026-01"                                  # part (a) fits
train_mask = (df.snapshot_month .>= "2025-09") .& (df.snapshot_month .<= "2026-01")  # parts (b), (c)

@assert !any(test_mask .& upto_jan) && !any(test_mask .& train_mask)
X_te_all, X_te, y_te = X_all[test_mask, :], X_kept[test_mask, :], y[test_mask]
```

```
println("training rows (Sep 2025–Jan 2026): ", sum(train_mask), " | test rows: ", sum(test_mask))
```

```
training rows (Sep 2025–Jan 2026): 16922
 | test rows: 8665
```

Unweighted OCT (`max_depth=3`, `minbucket=50`, `cp=0.001`) on all months up to Jan 2026

OCT comparison

Using all columns gives a root split of `last_invoice_usd < 23.455` and a test AUC of **0.9453**, while using only the retained columns gives a root split on `billing = monthly` and a test AUC of **0.8508**. The higher AUC of the all-column model is misleading because `last_invoice_usd` appears to contain information measured at extraction rather than at the historical snapshot. After removing this leakage and the other inappropriate features, the AUC decreases, but the retained-feature model gives a more credible measure of performance using information available at prediction time.

In [5]:
```
# Text description of the split at `node` (the condition for going to the LOWER child)
function describe_split(lnr, node)
    IAI.is_leaf(lnr, node) && return "leaf (no split)"
    f = IAI.get_split_feature(lnr, node)
    if IAI.is_parallel_split(lnr, node)
        return "$f < $(round(IAI.get_split_threshold(lnr, node), digits = 3))"
    elseif IAI.is_categoric_split(lnr, node) || IAI.is_ordinal_split(lnr, node)
        cats = IAI.get_split_categories(lnr, node)
        lower = sort([string(k) for (k, v) in cats if v])
        return "$f in {$(join(lower, ", "))}"
    else
        return "$f (non-parallel split)"
    end
end

oct_a_params = (random_seed = SEED, criterion = :gini, max_depth = 3, minbucket = 50, cp = 0.001)

# (1) every column except snapshot_month and churned_60d
oct_a_all = IAI.OptimalTreeClassifier(; oct_a_params...)
```

```
IAI.fit!(oct_a_all, X_all[upto_jan, :], y[upto_jan])

# (2) kept columns only
oct_a_kept = IAI.OptimalTreeClassifier(; oct_a_params...)
IAI.fit!(oct_a_kept, X_kept[upto_jan, :], y[upto_jan])

DataFrame(
    model      = ["(1) all columns", "(2) kept columns"],
    root_split = [describe_split(oct_a_all, 1), describe_split(oct_a_kept, 1)],
    test_auc   = round.([IAI.score(oct_a_all,  X_te_all, y_te, criterion = :auc),
                         IAI.score(oct_a_kept, X_te,     y_te, criterion = :auc)], digits = 4))
```

```
[ Warning: This copy of Interpretable AI software is for academic purposes only and not for commercia
l use.
```

Out[5]: 2×3 DataFrame

Row	model	root_split	test_auc
	String	String	Float64
1	(1) all columns	last_invoice_usd < 23.455	0.9453
2	(2) kept columns	billing in {monthly}	0.8508

Part (b)

Churners weight 5, stayers weight 1; training months Sep 2025 – Jan 2026; kept columns; `GridSearch` with default validation.

In [6]:
```
X_tr, y_tr = X_kept[train_mask, :], y[train_mask]
w_tr = ifelse.(y_tr .== 1, 5.0, 1.0)

cart_grid = IAI.GridSearch(
    IAI.OptimalTreeClassifier(random_seed = SEED, criterion = :gini, localsearch = false),
    max_depth = [2, 3, 4], minbucket = [25, 50, 100])          # cp tuned automatically
IAI.fit!(cart_grid, X_tr, y_tr, sample_weight = w_tr)
```

```
oct_grid = IAI.GridSearch(
    IAI.OptimalTreeClassifier(random_seed = SEED, criterion = :gini),
    max_depth = [2, 3, 4], minbucket = [25, 50, 100])          # cp tuned automatically
IAI.fit!(oct_grid, X_tr, y_tr, sample_weight = w_tr)

rf_grid = IAI.GridSearch(
    IAI.RandomForestClassifier(random_seed = SEED),
    max_depth = [5, 10, 15], minbucket = [1, 25, 100])
IAI.fit!(rf_grid, X_tr, y_tr, sample_weight = w_tr)

xgb_grid = IAI.GridSearch(
    IAI.XGBoostClassifier(random_seed = SEED),
    num_round = [50, 100, 200], max_depth = [2, 4, 6])
IAI.fit!(xgb_grid, X_tr, y_tr, sample_weight = w_tr)
nothing
```

In [7]:

```
# (5 FN + FP) / n, in months of revenue
function cost_per_subscriber(pred, y)
    FN = sum((pred .== 0) .& (y .== 1))
    FP = sum((pred .== 1) .& (y .== 0))
    return (5FN + FP) / length(y), FN, FP
end

fmt_params(p) = join(["$k = $(v isa AbstractFloat ? round(v, sigdigits = 3) : v)"
                      for (k, v) in sort(collect(p), by = x -> string(first(x)))], ", ")

grids = ["CART" => cart_grid, "OCT" => oct_grid, "Random forest" => rf_grid, "XGBoost" => xgb_grid]

res_b = DataFrame(model = String[], selected_params = String[], test_auc = Union{Float64, Missing}[],
                  FN = Int[], FP = Int[], cost_per_subscriber = Float64[])
for (name, g) in grids
    lnr  = IAI.get_learner(g)
    pred = IAI.predict(lnr, X_te)
    c, FN, FP = cost_per_subscriber(pred, y_te)
    push!(res_b, (name, fmt_params(IAI.get_best_params(g)),
                  IAI.score(lnr, X_te, y_te, criterion = :auc), FN, FP, c))
end

# Baselines
```

```
n_te = length(y_te)
c0, FN0, FP0 = cost_per_subscriber(zeros(Int, n_te), y_te)
c1, FN1, FP1 = cost_per_subscriber(ones(Int, n_te), y_te)
push!(res_b, ("Baseline: no offers",  "-", missing, FN0, FP0, c0))
push!(res_b, ("Baseline: offer everyone", "-", missing, FN1, FP1, c1))

res_b.test_auc = map(x -> ismissing(x) ? x : round(x, digits = 4), res_b.test_auc)
res_b.cost_per_subscriber = round.(res_b.cost_per_subscriber, digits = 4)
res_b
```

Out[7]: 6×6 DataFrame

Row	model	selected_params	test_auc	FN	FP	cost_per_subscriber
	String	String	Float64?	Int64	Int64	Float64
1	CART	cp = 3.36e-5, max_depth = 4, minbucket = 100	0.8437	88	2330	0.3197
2	OCT	cp = 0.000498, max_depth = 4, minbucket = 25	0.8787	249	1317	0.2957
3	Random forest	max_depth = 15, minbucket = 25	0.8798	177	1666	0.2944
4	XGBoost	max_depth = 2, num_round = 100	0.8796	177	1695	0.2977
5	Baseline: no offers	-	*missing*	1123	0	0.648
6	Baseline: offer everyone	-	*missing*	0	7542	0.8704

Leaf threshold check (selected OCT)

In [8]:
```
oct = IAI.get_learner(oct_grid)
leaf_tab = combine(groupby(DataFrame(leaf = IAI.apply(oct, X_tr), churned = y_tr), :leaf),
                   :churned => mean => :churn_share)
leaf_tab.predicted = [IAI.get_classification_label(oct, l) for l in leaf_tab.leaf]
println("lowest training churn share among leaves predicting churn: ", r3(minimum(leaf_tab.churn_shar
println("highest training churn share among leaves predicting stay: ", r3(maximum(leaf_tab.churn_shar
println("1/6 = ", r3(1/6))
```

```
lowest training churn share among leaves predicting churn: 0.176
highest training churn share among leaves predicting stay: 0.151
1/6 = 0.167
```

Which term of the OCT objective does the 5:1 cost change?

The 5:1 weighting changes the **misclassification-loss term** of the OCT objective. This is because misclassifying a churner receives five times the weight of misclassifying a non-churner. It does not change the tree-complexity penalty.

Above what share of churners does a leaf predict churn?

A leaf predicts churn when the churn share exceeds $\frac{1}{6} \approx 0.167$. With churners weighted 5 and non-churners weighted 1, predicting churn is preferred when $5p > 1 - p$, which gives $p > 1/6$. This is consistent with the fitted OCT, where we found out that the lowest churn share among leaves predicting churn is 0.176, while the highest among leaves predicting stay is 0.151.

What does `GridSearch` hold out for validation?

By default, `GridSearch` creates its validation set by holding out a portion of the training data rather than respecting the chronological ordering of the snapshot months. The remaining observations are used to fit each candidate model, while the held-out observations are used to compare the different hyperparameter choices and select the best one. Thus, observations from the same training months can appear in both the fitting and validation sets. For example, some January 2026 observations could be used to fit a candidate model while other January 2026 observations are used to evaluate it.

Is that the right choice for this data?

Not ideally. Since the goal is to predict churn in future months, validation should also mimic predicting the future. A more appropriate approach is to train on Sep--Dec 2025 and use Jan 2026 for validation, before evaluating the final model on the Feb--Apr 2026 test set. Using this time-based validation gave a test AUC of 0.8788 and cost per subscriber of 0.2953, which are nearly identical to the default `GridSearch` results of 0.8787 and 0.2957. Thus, although time-based validation is more appropriate, the choice of validation method does not meaningfully change our results.

In [ ]:
```
# Robustness check: tune the same OCT grid on Sep–Dec 2025 and validate on Jan 2026 (no test data use
fit_mask = (df.snapshot_month .>= "2025-09") .& (df.snapshot_month .<= "2025-12")
val_mask = df.snapshot_month .== "2026-01"
```

```
w_of(v) = ifelse.(v .== 1, 5.0, 1.0)

oct_grid_time = IAI.GridSearch(
    IAI.OptimalTreeClassifier(random_seed = SEED, criterion = :gini),
    max_depth = [2, 3, 4], minbucket = [25, 50, 100])
IAI.fit!(oct_grid_time,
         X_kept[fit_mask, :], y[fit_mask], X_kept[val_mask, :],4 y[val_mask];
         sample_weight = (w_of(y[fit_mask]), w_of(y[val_mask])))   # (training weights, validation we

oct_time = IAI.get_learner(oct_grid_time)
c_time, _, _ = cost_per_subscriber(IAI.predict(oct_time, X_te), y_te)
println("time-based validation (Jan 2026): ", fmt_params(IAI.get_best_params(oct_grid_time)),
        " | test AUC = ", round(IAI.score(oct_time, X_te, y_te, criterion = :auc), digits = 4),
        " | cost = ", round(c_time, digits = 4))
```

```
time-based validation (Jan 2026): cp = 4.8e-5, max_depth = 4, minbucket = 25 | test AUC = 0.8788 | co
st = 0.2953
```

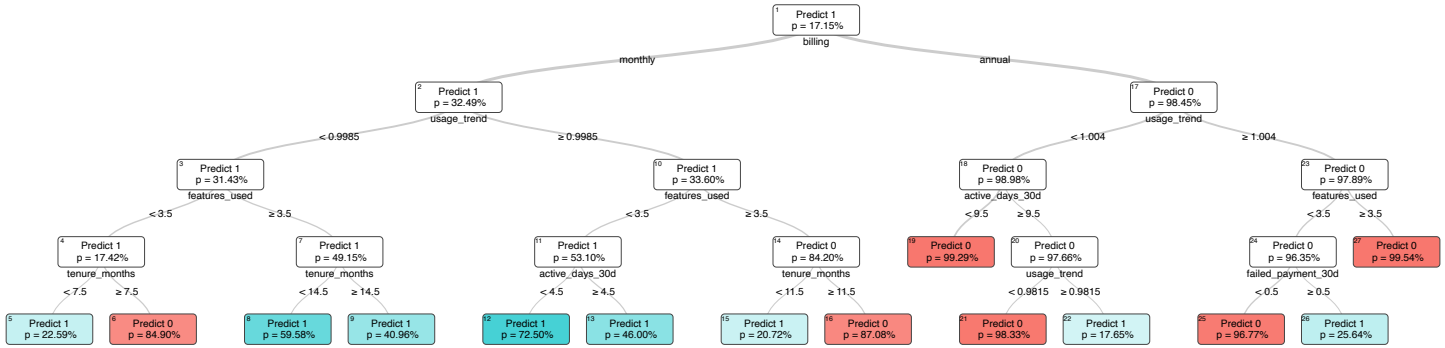
Part (c)

Deployed tree

In [10]:

```
deploy = oct   # selected OCT from part (b)
IAI.write_html("deployed_tree.html", deploy)
deploy
```

Out[10]: Collapse Expand Save PNG



Who gets an offer

Offer policy. The deployed OCT sends an offer whenever a subscriber reaches a leaf that predicts churn (`Predict 1`). The table below translates each churn-predicting leaf into its corresponding decision rule and reports the churn rate within that group. On the test set, the policy sends offers to **25.3%** of subscribers while reaching **77.8%** of subscribers who actually churn. Its resulting cost is **0.2957 per subscriber**. Most offer rules apply to monthly subscribers and use combinations of usage trend, features used, tenure, and recent activity, while a small number of annual subscribers are targeted based on usage and failed-payment behavior.

In [11]:

```
# Collapse each path to a churn leaf into one bound per feature
function leaf_rules(lnr)
    rules = Dict{Int, Dict{Symbol, Any}}()
    function walk(node, conds)
        if IAI.is_leaf(lnr, node)
            rules[node] = conds
            return
        end
        f = IAI.get_split_feature(lnr, node)
        lo, hi = copy(conds), copy(conds)
        if IAI.is_parallel_split(lnr, node)
            t = IAI.get_split_threshold(lnr, node)
            a, b = get(conds, f, (-Inf, Inf))
            lo[f] = (a, min(b, t)); hi[f] = (max(a, t), b)
        else
            cats = IAI.get_split_categories(lnr, node)
            allowed = get(conds, f, Set(string.(keys(cats))))
            lo[f] = intersect(allowed, Set(string(k) for (k, v) in cats if v))
            hi[f] = intersect(allowed, Set(string(k) for (k, v) in cats if !v))
        end
        walk(IAI.get_lower_child(lnr, node), lo)
        walk(IAI.get_upper_child(lnr, node), hi)
    end
    walk(1, Dict{Symbol, Any}())
    return rules
end

function fmt_rule(conds)
    parts = String[]
    for f in sort(collect(keys(conds)), by = string)
        v = conds[f]
        if v isa Tuple
            a, b = v
            isfinite(a) && isfinite(b) ? push!(parts, "$(round(a, digits = 3)) ≤ $f < $(round(b, digi
            isfinite(a)                ? push!(parts, "$f ≥ $(round(a, digits = 3))") :
                                         push!(parts, "$f < $(round(b, digits = 3))")
        else
            push!(parts, "$f = $(join(sort(collect(v)), "/"))")
```

```
        end
    end
    return join(parts, ", ")
end

rules   = leaf_rules(deploy)
leaf_te = IAI.apply(deploy, X_te)
leaf_tr = IAI.apply(deploy, X_tr)
churn_leaves = sort([l for l in keys(rules) if IAI.get_classification_label(deploy, l) == 1])
offers = DataFrame(
    rule               = [fmt_rule(rules[l]) for l in churn_leaves],
    train_churn_share  = [r3(mean(y_tr[leaf_tr .== l])) for l in churn_leaves],
    test_rows          = [sum(leaf_te .== l) for l in churn_leaves],
    test_churn_share   = [r3(mean(y_te[leaf_te .== l])) for l in churn_leaves])
sort!(offers, :test_rows, rev = true)

pred_te = IAI.predict(deploy, X_te)
c, FN, FP = cost_per_subscriber(pred_te, y_te)
println("test: offer rate = ", pct(mean(pred_te .== 1)), " | churners caught = ", pct(1 - FN / sum(y_
        " | cost per subscriber = ", round(c, digits = 4))
offers
```

```
test: offer rate = 25.3% | churners caught = 77.8% | cost per subscriber = 0.2957
```

Out[11]: 8×4 DataFrame

Row	rule	train_churn_share	test_rows	test_churn_share
	String	Float64	Int64	Float64
1	active_days_30d ≥ 4.5, billing = monthly, features_used < 3.5, usage_trend ≥ 0.998	0.46	517	0.505
2	billing = monthly, features_used < 3.5, tenure_months < 7.5, usage_trend < 0.998	0.226	419	0.239
3	billing = monthly, features_used ≥ 3.5, tenure_months < 11.5, usage_trend ≥ 0.998	0.207	378	0.23
4	billing = monthly, features_used ≥ 3.5, tenure_months ≥ 14.5, usage_trend < 0.998	0.41	349	0.358
5	billing = monthly, features_used ≥ 3.5, tenure_months < 14.5, usage_trend < 0.998	0.596	305	0.564
6	active_days_30d < 4.5, billing = monthly, features_used < 3.5, usage_trend ≥ 0.998	0.725	181	0.669
7	active_days_30d ≥ 9.5, billing = annual, 0.981 ≤ usage_trend < 1.004	0.176	21	0.0
8	billing = annual, failed_payment_30d ≥ 0.5, features_used < 3.5, usage_trend ≥ 1.004	0.256	21	0.381

Age-policy checks

Age-policy audit. We check whether the retention policy could target subscribers based on age either directly or through a proxy. First, we verify that the deployed tree does not split on `age_band` or `signup_channel`, which we excluded because `signup_channel` was strongly associated with age. Second, we test whether age 18--24 can be predicted from the retained features. Their AUC is only **0.5485**, compared with **0.792** using `signup_channel` alone, suggesting that removing `signup_channel` substantially reduces age information available to the model. Finally, we compare offer rates across age groups in the test set; these range from **23.3% to 26.5%**, with no large differences across age bands.

In [12]:

```
# 1. Features the deployed tree splits on
used = unique(string.([IAI.get_split_feature(deploy, t) for t in 1:IAI.get_num_nodes(deploy) if !IAI.
println("1. tree splits on: ", join(used, ", "), " | uses age_band or signup_channel: ",
        any(in(("age_band", "signup_channel")), used))

# 2. Can age 18-24 be predicted? (kept features vs signup_channel alone)
young = df.age_band .== "18-24"
proxy_tree = IAI.OptimalTreeClassifier(random_seed = SEED, criterion = :gini, max_depth = 3, minbucke
IAI.fit!(proxy_tree, X_kept[train_mask, :], young[train_mask])
X_chan = make_X(df, ["signup_channel"])
proxy_chan = IAI.OptimalTreeClassifier(random_seed = SEED, criterion = :gini, max_depth = 3, minbucke
IAI.fit!(proxy_chan, X_chan[train_mask, :], young[train_mask])
println("2. test AUC for predicting age 18-24: kept features = ",
        round(IAI.score(proxy_tree, X_te, young[test_mask], criterion = :auc), digits = 4),
        ", signup_channel alone = ",
        round(IAI.score(proxy_chan, X_chan[test_mask, :], young[test_mask], criterion = :auc), digits

# 3. Offer rate by age band on the test set (age used only for this audit)
println("3. offer rate by age band (test):")
te = df[test_mask, [:age_band, :churned_60d]]
te.offer = pred_te .== 1
sort(combine(groupby(te, :age_band),
    nrow         => :n,
    :churned_60d => (x -> r3(mean(x))) => :churn_rate,
    :offer       => (x -> r3(mean(x))) => :offer_rate), :age_band)
```

```
1. tree splits on: billing, usage_trend, features_used, tenure_months, active_days_30d, failed_paymen
t_30d | uses age_band or signup_channel: false
2. test AUC for predicting age 18-24: kept features = 0.5485, signup_channel alone = 0.792
3. offer rate by age band (test):
```

Out[12]: 5×4 DataFrame

Row	age_band	n	churn_rate	offer_rate
	String7	Int64	Float64	Float64
1	18-24	1629	0.136	0.255
2	25-34	3056	0.132	0.261
3	35-44	2020	0.119	0.233
4	45-54	1117	0.136	0.265
5	55+	843	0.123	0.251

Board bullets

- **Model to deploy:** We recommend deploying the weighted **OCT**. It achieves a test AUC of **0.8787** and a cost of **0.2957 per subscriber**, compared with an AUC of 0.8798 and cost of 0.2944 for the best-performing random forest. Although the random forest performs slightly better numerically, the difference in cost is only **0.0013 per subscriber**. The OCT therefore provides nearly the same predictive and economic performance while giving us a single interpretable decision tree whose offer decisions can be explained and audited.
- **Subscribers receiving offers:** We send a free-month offer to any subscriber who reaches a leaf of the OCT that predicts churn. On the test set, this policy sends offers to **25.3% of subscribers** and captures **77.8% of subscribers who actually churn**. Most of the targeted groups are monthly subscribers, with the tree further distinguishing them using usage trend, number of features used, tenure, and recent activity. A small number of annual subscribers are also targeted based on combinations of usage behavior and failed payments. This allows us to target the retention offer rather than offering it to every subscriber.
- **Age-policy check:** We excluded `age_band` from the model and also removed `signup_channel` because it was a strong proxy for age. We verified that neither variable appears in the deployed tree. We then checked whether the retained features could still indirectly predict whether a subscriber was age 18--24; the resulting test AUC was only **0.5485**, compared with **0.792** using `signup_channel` alone. Finally, we audited the actual offer

rates by age group, which ranged from **23.3% to 26.5%**. Together, these checks provide evidence that the deployed policy is not strongly dependent on age.